

课程表管理系统数据库设计报告

王亦玮

学号：2351274

2025 年 12 月 27 日

摘要

本报告详细阐述了基于 Vue 3 + TypeScript + Flask + SQLite 技术栈的课程表管理系统数据库设计全过程。系统融合了 RAG（检索增强生成）技术与混合搜索架构，实现了智能课程推荐、可视化课表管理、用户认证与 AI 交互等功能。报告从需求分析出发，逐步完成概念设计、逻辑设计与物理设计，并对课程数据处理机制、RAG 实现原理、向量检索技术、安全机制与性能优化进行了深入探讨。报告包含详细的 SQL 表结构设计、数据类型选择、索引策略以及具体的代码实现，符合软件工程规范，内容详实，满足课程设计报告要求。

关键词：课程管理系统；数据库设计；RAG；向量检索；混合搜索；SQLite；Vue 3；Flask；JWT

目录

1	引言	1
1.1	项目概述	1
1.2	报告目的与范围	1
1.3	项目背景与意义	1
2	需求分析	2
2.1	功能需求分析	2
2.1.1	用户管理模块	2
2.1.2	课程管理模块	2
2.1.3	课表管理模块	2
2.1.4	AI 交互模块	2
2.1.5	用户偏好管理模块	3
2.2	非功能需求分析	3
2.2.1	性能需求	3
2.2.2	安全需求	3
2.2.3	可扩展性需求	3
2.3	数据流分析	4
3	数据库概念设计	4
3.1	实体识别	4
3.1.1	用户实体 (User)	4
3.1.2	课表实体 (Timetable)	4
3.1.3	AI 聊天历史实体 (AI_Chat_History)	5
3.1.4	用户收藏实体 (User_Favorites)	5
3.1.5	课程实体 (Course)	5
3.2	实体关系分析	5
3.3	E-R 图设计	6
4	数据库逻辑设计	6
4.1	关系模式设计	6
4.1.1	用户表 (users)	6
4.1.2	课表表 (timetables)	7
4.1.3	AI 聊天历史表 (ai_chat_history)	7
4.1.4	用户收藏表 (user_favorites)	7
4.2	数据完整性约束	7
4.2.1	实体完整性	8
4.2.2	参照完整性	8
4.2.3	用户定义完整性	8
4.3	关系规范化分析	8
4.3.1	第一范式 (1NF)	9
4.3.2	第二范式 (2NF)	9
4.3.3	第三范式 (3NF)	9
5	课程数据处理机制	9
5.1	CSV 数据结构分析	9
5.2	数据解析机制详解	10
5.3	内存中数据结构设计	10

5.4	课程数据加载与管理流程	11
6	数据库物理设计	11
6.1	表结构实现与 SQL 语句	11
6.2	索引设计策略与优化	13
6.3	存储参数与引擎选择	15
6.4	安全性设计深入	15
7	RAG 与向量检索技术详解	16
7.1	RAG 技术原理与在本系统的应用	16
7.2	向量检索技术的实现细节	17
7.2.1	向量化模型选择与处理	17
7.2.2	相似度计算与搜索	18
7.2.3	性能优化考虑	18
7.3	三层混合搜索架构的融合策略	18
7.3.1	并行执行与结果获取	19
7.3.2	结果融合算法	19
7.4	RAG 上下文构建与提示词工程	19
7.4.1	动态上下文构建	19
7.4.2	系统提示词设计	20
8	系统功能实现与数据交互	20
8.1	用户管理功能的实现	21
8.2	课程搜索功能的实现	21
8.3	课表管理功能的实现	21
8.4	AI 交互功能的实现	22
9	数据库优化策略与性能调优	23
9.1	查询优化实践	23
9.2	存储优化策略	23
9.3	性能监控与调优建议	23
10	数据库维护、备份与安全	24
10.1	备份策略与实施	24
10.2	恢复流程与应急预案	24
10.3	数据迁移与版本管理	25
10.4	安全加固措施	25
11	系统架构与技术栈总结	25
11.1	前端技术栈详解	25
11.2	后端技术栈详解	26
11.3	数据流处理全景	26
12	总结与展望	27
12.1	项目成果总结	27
12.1.1	设计亮点回顾	27
12.1.2	数据库设计价值	27
12.2	挑战与解决方案	28
12.3	未来展望与扩展方向	28

12.4 结语	29
-------------------	----

1 引言

1.1 项目概述

课程表管理系统是一款面向高校师生的智能化教学辅助平台，旨在解决传统课程管理中存在的搜索效率低、课表规划难、推荐不精准等问题。系统采用前后端分离架构，前端基于 Vue 3 + TypeScript 构建响应式用户界面，后端使用 Python Flask 提供 RESTful API 与 AI 服务编排，数据库选用轻量级 SQLite 存储用户相关数据，课程数据则通过 CSV 文件加载至内存实现高速检索。该系统不仅仅是一个简单的课程信息存储和展示工具，更是一个融合了现代人工智能技术的智能教学助手。

系统的核心创新在于引入 RAG（检索增强生成）技术与三层混合搜索架构，使得系统不仅支持传统关键词搜索，还具备语义理解与 AI 联想能力，能够根据用户当前课表、空闲时间等上下文信息，提供个性化、可操作的选课建议，实现了从传统教务系统到 AI 教学助手的跃迁。系统通过精心设计的数据库结构，将课程数据、用户信息、交互记录等有机整合，为前端提供高效的数据服务。

1.2 报告目的与范围

本报告旨在系统性地展示课程表管理系统数据库设计的完整流程，涵盖需求分析、概念设计、逻辑设计、物理设计等阶段，确保数据库结构合理、性能高效、扩展性强，并为后续系统开发与维护提供清晰的技术文档支持。报告将详细讨论数据库设计的每一个关键决策，包括实体识别、关系建立、约束设置、索引策略以及安全性考虑，同时深入探讨与课程数据处理、AI 技术集成相关的特殊设计考量。

1.3 项目背景与意义

随着高校课程体系日趋复杂，课程数量不断增加，传统的纸质课表或简易电子表格已难以满足师生在课程查询、排课规划、智能推荐等方面的需求。学生常常面临课程信息分散、搜索效率低下、课表冲突难以避免等问题。本系统应运而生，通过整合现代 Web 技术、AI 大模型与向量检索技术，构建了一个既能管理课程信息，又能提供智能交互的教学辅助平台，极大地提升了选课与排课的效率与体验。

2 需求分析

2.1 功能需求分析

2.1.1 用户管理模块

用户管理模块是系统的基础，需要支持完整的用户生命周期管理。用户注册功能需要收集邮箱、密码、姓名等基本信息，其中邮箱作为唯一标识，密码需经过安全的哈希算法处理后再存储。用户登录功能需要验证用户身份，成功后生成并返回 JWT（JSON Web Token）用于后续请求的认证。用户资料管理允许用户查看和修改个人信息，包括姓名和头像等。此外，系统还应提供密码重置功能，通过邮箱验证机制确保账户安全。

2.1.2 课程管理模块

课程管理模块负责课程数据的存储、检索和推荐。课程数据来源于外部的 CSV 文件，系统需要解析并加载这些数据到内存中，以支持高效的检索操作。课程搜索功能需要支持多种模式：精确的关键字搜索用于快速定位已知课程；语义理解搜索能够理解用户的自然语言查询意图；AI 联想搜索则通过大语言模型扩展用户的查询关键词，提供更全面的结果。课程推荐功能需要结合用户的历史行为和当前课表，提供个性化的课程建议。

2.1.3 课表管理模块

课表管理模块允许用户创建、保存和管理多个课表。每个课表可以包含多门课程，用户可以为课表命名并添加描述。课表可视化功能需要将课程信息以网格形式展示，清晰显示每门课的上课时间、地点和教师。系统需要实时检测课程时间冲突，并在界面上给出直观的提示。课表数据需要在用户操作后自动保存，确保数据不会丢失。

2.1.4 AI 交互模块

AI 交互模块是系统的智能核心，通过集成大语言模型提供对话式课程咨询服务。系统需要记录用户与 AI 的完整对话历史，包括用户的问题和 AI 的回答，以便进行上下文管理和历史回顾。AI 助手需要能够理解用户关于课程安排、时间冲突、课程推荐等

问题的自然语言描述，并给出准确、有用的回答。更重要的是，AI 的回答应当结合用户的实时数据（如当前课表、空闲时间等），提供个性化的建议。

2.1.5 用户偏好管理模块

用户偏好管理模块允许用户收藏感兴趣的课程，方便日后快速访问和选课。收藏的课程需要与用户账户关联，并且每个用户对每门课程只能收藏一次，避免数据冗余。系统还应支持对收藏课程进行分类和标签管理，但作为基础版本，可以先实现简单的列表展示。

2.2 非功能需求分析

2.2.1 性能需求

系统需要在高并发场景下保持稳定和高效。数据库查询的响应时间应控制在 100 毫秒以内，特别是在用户进行课程搜索和课表加载时。AI 交互的响应时间虽然可以稍长，但也应尽量优化，确保用户体验的流畅性。课程数据的加载和解析需要在系统启动时快速完成，避免用户等待。系统需要能够支持至少 100 个用户同时在线操作。

2.2.2 安全需求

用户密码必须使用强哈希算法（如 bcrypt）加密存储，绝对不能以明文形式出现在数据库中。所有 API 接口需要进行身份验证，未经验证的请求应被拒绝。JWT 令牌需要设置合理的有效期，并支持刷新机制。用户只能访问自己的数据，不能越权访问其他用户的课表、收藏或聊天记录。系统还需要防范常见的网络攻击，如 SQL 注入、跨站脚本攻击（XSS）等。

2.2.3 可扩展性需求

数据库结构设计应具备良好的可扩展性，以适应未来可能增加的新功能。例如，未来可能需要增加课程评价、教师评分、课程论坛等功能，数据库应能够在不进行大规模重构的情况下支持这些扩展。系统应采用模块化设计，不同功能模块之间耦合度低，便于独立升级和维护。数据备份和恢复机制也是可扩展性的重要方面，确保系统数据的安全性。

2.3 数据流分析

系统的数据流可以分为四个主要流程：用户注册登录流程、课程搜索流程、课表管理流程和 AI 交互流程。用户注册登录流程中，前端收集用户输入，后端验证数据合法性，将用户信息存入数据库，并返回认证结果。课程搜索流程中，用户输入查询条件，后端执行混合搜索算法，从内存中的课程数据检索结果，按相关性排序后返回。课表管理流程中，用户在前端操作课表（添加、删除课程），前端将更新后的课表数据发送到后端，后端验证后存入数据库。AI 交互流程中，用户提出问题，后端首先检索用户的相关上下文数据（如课表），然后将问题和上下文一起发送给 AI 模型，得到回答后存储对话记录并返回给用户。

3 数据库概念设计

3.1 实体识别

3.1.1 用户实体 (User)

用户实体是系统的核心实体，代表注册并使用系统的个体。其主要属性包括：ID（唯一标识符，通常为自增整数）、邮箱（用于登录和通信，具有唯一性）、密码哈希（存储加密后的密码，而非明文）、姓名（用户的真实姓名或昵称）、头像 URL（指向用户头像图片的链接，可选）、创建时间（记录用户注册的时间戳）。用户实体是其他多个实体的所有者或关联者。

3.1.2 课表实体 (Timetable)

课表实体代表用户创建的一个具体课表安排。其主要属性包括：ID（唯一标识符）、用户 ID（外键，关联到创建此课表的用户）、名称（用户为课表起的名字）、描述（对课表的简要说明，可选）、课程信息（以 JSON 格式存储该课表包含的所有课程详情，包括课程 ID、名称、时间、地点等）、创建时间（记录课表创建的时间戳）。一个用户可以拥有多个课表，每个课表代表一个不同的课程安排方案。

3.1.3 AI 聊天历史实体 (AI_Chat_History)

AI 聊天历史实体记录用户与 AI 助手的每一次对话消息。其主要属性包括：ID（唯一标识符）、用户 ID（外键，关联到发起对话的用户）、会话 ID（用于区分同一用户的不同对话会话，例如不同主题的咨询）、消息角色（标明该条消息是来自“user”还是“assistant”）、消息内容（对话的文本内容）、时间戳（消息发送的时间）。该实体用于保存完整的对话上下文，支持多轮对话和历史回顾。

3.1.4 用户收藏实体 (User_Favorites)

用户收藏实体记录用户标记为感兴趣的课程。其主要属性包括：ID（唯一标识符）、用户 ID（外键，关联到收藏课程的用户）、课程 ID（被收藏课程的唯一标识符，来自课程数据）、课程名称（被收藏课程的名称，用于快速显示）、课程信息（以 JSON 格式存储课程的详细信息，便于直接使用）、创建时间（收藏操作的时间戳）。该实体实现了用户与课程之间的多对多收藏关系。

3.1.5 课程实体 (Course)

课程实体代表学校开设的一门具体课程。需要特别说明的是，在本系统中，课程数据主要来源于外部的 CSV 文件，为了追求极致的检索性能，课程数据在系统启动时被加载到内存中，而不是存储在关系型数据库里。其内存中的数据结构包括：ID（课程唯一标识符）、名称（课程名称）、排课信息（原始的排课字符串）、教师信息（授课教师）、上课时间地点（解析后的结构化信息）。课程实体与其他实体的关联是通过 ID 引用实现的。

3.2 实体关系分析

用户实体与课表实体之间是一对多的关系。一个用户可以创建并保存多个课表，而一个课表只属于一个特定的用户。这种关系通过课表实体中的 `user_id` 外键来实现。为了保持数据一致性，当用户被删除时，其所有的课表记录也应该被自动删除，这可以通过数据库的外键约束中的 `ON DELETE CASCADE` 规则来实现。

用户实体与 AI 聊天历史实体之间也是一对多的关系。一个用户可以进行多次 AI 对话，产生多条聊天记录，而每条聊天记录只属于一个用户。同样通过 `user_id` 外键关

联，并设置级联删除。

用户实体与课程实体之间通过用户收藏实体建立了多对多的关系。一个用户可以收藏多门课程，一门课程也可以被多个用户收藏。这种关系通过一个中间表（用户收藏表）来实现，该表包含用户 ID 和课程 ID 的组合作为唯一约束，确保用户不会重复收藏同一门课程。

课程实体与课表实体之间也存在一种间接的关联。课表实体中的 `courses` 字段（JSON 格式）包含了该课表中所有课程的 ID 和详细信息。这种设计避免了在关系型数据库中建立复杂的多对多关系表，提高了查询效率，但牺牲了部分关系数据库的规范化特性。

3.3 E-R 图设计

虽然本报告无法直接嵌入图形，但可以通过文字描述 E-R 图的主要结构。系统的 E-R 图以用户实体为中心，向外辐射出三条关系线：一条指向课表实体（一对多），一条指向 AI 聊天历史实体（一对多），一条通过用户收藏实体指向课程实体（多对多）。课程实体作为相对独立的实体，与课表实体通过 JSON 字段内的引用建立关联。所有关系都标注了基数（1:N 或 N:M），并在需要的地方标注了级联删除的约束。

4 数据库逻辑设计

4.1 关系模式设计

根据概念设计，我们将实体和关系转化为具体的关系模式（即数据库表结构）。所有表名均使用英文小写，字段名使用下划线分隔的蛇形命名法，以提高可读性和跨平台兼容性。

4.1.1 用户表 (users)

用户表存储所有注册用户的基本信息。其关系模式定义为：

```
1 Users(ID, email, password_hash, full_name, avatar_url, created_at)
```

其中 ID 为整型主键，自动递增；email 为变长字符串，具有唯一性约束；password_hash

为存储密码哈希值的字符串；full_name 为用户姓名；avatar_url 为可选的头像链接；created_at 为记录创建时间的时间戳。

4.1.2 课表表 (timetables)

课表表存储用户创建的所有课表。其关系模式定义为：

```
1 Timetables(ID, user_id, name, description, courses, created_at)
```

其中 ID 为整型主键，自动递增；user_id 为外键，引用 users 表的 ID 字段；name 为课表名称；description 为可选的课表描述；courses 为以 JSON 文本格式存储的课程列表；created_at 为创建时间。

4.1.3 AI 聊天历史表 (ai_chat_history)

AI 聊天历史表存储用户与 AI 的所有对话记录。其关系模式定义为：

```
1 AI_Chat_History(ID, user_id, session_id, message_role, message_content,  
    timestamp)
```

其中 ID 为整型主键，自动递增；user_id 为外键，引用 users 表的 ID 字段；session_id 为区分不同对话会话的字符串；message_role 为标识发送者角色的字符串（'user' 或 'assistant'）；message_content 为消息的文本内容；timestamp 为消息发送的时间。

4.1.4 用户收藏表 (user_favorites)

用户收藏表存储用户的课程收藏记录。其关系模式定义为：

```
1 User_Favorites(ID, user_id, course_id, course_name, course_info, created_at  
    )
```

其中 ID 为整型主键，自动递增；user_id 为外键，引用 users 表的 ID 字段；course_id 为被收藏课程的原始 ID（字符串）；course_name 为课程名称；course_info 为以 JSON 格式存储的课程详细信息（便于快速展示）；created_at 为收藏时间。

4.2 数据完整性约束

为了保证数据库中数据的准确性和一致性，我们设计了以下完整性约束：

4.2.1 实体完整性

每个表都必须有一个主键，且主键的值不能为 NULL，必须唯一。在我们的设计中，所有表的主键都是名为'id' 的 INTEGER 字段，并通过 AUTOINCREMENT 关键字确保其唯一性和自增性。

4.2.2 参照完整性

所有外键字段(如 timetables.user_id, ai_chat_history.user_id, user_favorites.user_id)都必须引用其父表(users 表)中实际存在的 ID 值。在 SQLite 中，我们通过 FOREIGN KEY 约束来定义。此外，为了保持数据的一致性，我们为所有外键约束添加了 ON DELETE CASCADE 子句。这意味着当 users 表中的某个用户记录被删除时，数据库中所有引用了该用户 ID 的相关记录(课表、聊天历史、收藏)都会被自动删除，防止出现孤儿数据。

4.2.3 用户定义完整性

除了数据库本身提供的约束，我们还根据业务需求定义了一些特定的约束：

- **邮箱唯一性**：在 users 表中，email 字段被设置为 UNIQUE，确保一个邮箱只能注册一个账户。
- **字段非空**：关键业务字段如 email, password_hash, full_name 等都设置为 NOT NULL，确保数据的完整性。
- **收藏唯一性**：在 user_favorites 表中，我们设置了 UNIQUE(user_id, course_id) 约束，确保同一个用户不能重复收藏同一门课程。
- **角色枚举**：在 ai_chat_history 表中，message_role 字段虽然没有在数据库层做枚举约束，但在应用层逻辑中，它只允许是'user' 或'assistant' 两个值之一。

4.3 关系规范化分析

数据库设计遵循了关系数据库的规范化原则，以减少数据冗余和提高数据一致性。我们对所有表进行了分析，确认它们至少满足第三范式 (3NF)。

4.3.1 第一范式 (1NF)

第一范式要求表的每一列都是不可再分的原子值。我们设计的四个表中，所有字段存储的都是单一值。例如，`courses` 和 `course_info` 字段虽然存储的是 JSON 字符串，但它们是作为一个整体的文本值存储的，符合 1NF 的要求。

4.3.2 第二范式 (2NF)

第二范式要求非主属性完全依赖于主键，消除部分依赖。在我们的所有表中，主键都是单一列 (`id`)，不存在复合主键，因此也就不存在非主属性部分依赖于主键的问题，自然满足 2NF。

4.3.3 第三范式 (3NF)

第三范式要求消除传递依赖，即任何非主属性不能依赖于其他非主属性。在我们的设计中，所有非主属性都直接依赖于其所在表的主键 `id`。例如，在 `timetables` 表中，`name`、`description`、`courses` 都直接依赖于课表 ID，而 `user_id` 作为外键虽然也依赖于 `users` 表的主键，但这属于不同表之间的关系，并不违反 3NF。因此，我们的设计满足第三范式。

5 课程数据处理机制

5.1 CSV 数据结构分析

课程数据来源于一个名为 `filtered_courses.csv` 的外部文件。该文件采用逗号分隔值格式，包含以下三列：

- 新课程序号**：课程的唯一标识符，通常由字母和数字组成，如“CS101”。
- 课程名称**：课程的中文名称，如“数据结构与算法”。
- 排课信息**：一个复杂的字符串，包含了该课程的所有上课时间、地点、周次和教师信息。其格式示例为：“张老师 (1234567) 星期一 5-7 节 [1-16] 二教 101 李老师 (7654321) 星期三 3-4 节 [1-8] 三教 201”。这个字符串需要被解析才能提取出结构化的信息。

5.2 数据解析机制详解

由于排课信息字符串结构复杂且不规则,我们设计了一个专门的解析函数 `parse_schedule_string`。该函数的核心思路是利用正则表达式进行模式匹配和提取。

解析步骤:

1. **预处理:** 检查输入字符串是否为空或仅包含空白字符,若是则返回空列表。
2. **分割:** 使用正则表达式 `r'(星期 [一二三四五六日])'` 将字符串按星期几进行分割。
例如,字符串“张老师... 星期一... 星期三...”会被分割成 `[' 张老师...', ' 星期一', '... 李老师...', ' 星期三', '...']`。
3. **遍历匹配:** 以步长为 2 遍历分割后的列表,每次取出一对“星期几”和“后续信息段”。
4. **提取核心信息:** 对每个“后续信息段”,使用正则表达式 `r'\s*(\d+)-(\d+) 节\s+\[(.*?)\]\s+([^\s]*)'` 来匹配教师姓名和可能的工号。
4. **提取教师信息:** 教师信息通常位于当前信息段之前的部分。通过另一个正则表达式 `r'([\s].*?)\d+\s*$'` 来匹配教师姓名和可能的工号。
5. **组装结果:** 将提取到的所有信息(星期、节次、周次、地点、教师)组装成一个字典对象,并添加到结果列表中。
6. **返回:** 返回包含所有上课时间段信息的列表。

这个解析函数具有较强的鲁棒性,能够处理教师信息缺失、多时间段、多教师等多种复杂情况,是系统能够正确理解课程安排的关键。

5.3 内存中数据结构设计

解析后的课程数据在内存中以 TypeScript 接口的形式定义,这确保了前后端数据类型的一致性。主要包含两个核心接口:

- **Session 接口:** 表示一个单一的上课时间段。包含星期(数字和字符串两种表示)、开始和结束节次、上课周次范围、具体上课地点和授课教师。
- **Course 接口:** 表示一门完整的课程。包含课程 ID、名称、用于显示的教师姓名、原始排课字符串以及一个由多个 Session 对象组成的数组(一门课可能有多周次、多时间段的安排)。

这种结构化的表示方式,使得课程数据可以在前端进行灵活的展示(如渲染到课表

网格中)，也便于后端进行各种计算和检索（如时间冲突检测、基于时间的课程推荐）。

5.4 课程数据加载与管理流程

课程数据的管理遵循以下流程：

1. **启动加载**：当后端服务启动时，自动读取位于指定路径的 `filtered_courses.csv` 文件。
2. **数据转换**：使用 `pandas` 库将 CSV 文件读入 `DataFrame`，便于进行行级操作。
3. **逐行解析**：遍历 `DataFrame` 的每一行，对“排课信息”列调用 `parse_schedule_string` 函数进行解析，生成结构化的 `Session` 列表。
4. **构建课程对象**：将解析结果与课程 ID、名称等结合，构建完整的 `Course` 对象。
5. **全局存储**：将所有 `Course` 对象存储在一个全局的数组或列表中（例如命名为 `ALL_COURSES`）。
6. **提供服务**：前端通过 RESTful API（如 `GET /api/courses`）请求课程数据时，后端直接将内存中的 `ALL_COURSES` 数据（或经过搜索处理后的子集）返回。
7. **向量化预处理**：为了提高语义搜索的速度，系统在启动时还会使用 `Transformer` 模型将每门课程的描述（或名称）转换为向量（`embedding`），并存储起来。当用户进行语义搜索时，只需计算查询向量与这些预存向量的相似度即可。

这种将课程数据常驻内存的设计，虽然会增加服务启动时的内存占用，但换来了极高的数据检索速度，非常适合课程数据相对稳定、读多写少的场景。

6 数据库物理设计

6.1 表结构实现与 SQL 语句

基于逻辑设计，我们编写了具体的 SQL DDL 语句来创建数据库表。选择 SQLite 作为数据库引擎，其语法简洁且能满足本项目需求。

```
1 -- 用户表
2 CREATE TABLE IF NOT EXISTS users (
3     id INTEGER PRIMARY KEY AUTOINCREMENT,
4     email VARCHAR(255) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     full_name VARCHAR(255) NOT NULL,
```

```
7     avatar_url VARCHAR(500),
8     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
9 );
10
11 -- 课表表
12 CREATE TABLE IF NOT EXISTS timetables (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     user_id INTEGER NOT NULL,
15     name VARCHAR(255) NOT NULL,
16     description TEXT,
17     courses TEXT NOT NULL, -- JSON格式存储课程信息
18     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
19     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
20     -- 可选：为用户ID添加索引以加速按用户查询
21     CREATE INDEX idx_timetables_user_id ON timetables(user_id)
22 );
23
24 -- AI聊天历史表
25 CREATE TABLE IF NOT EXISTS ai_chat_history (
26     id INTEGER PRIMARY KEY AUTOINCREMENT,
27     user_id INTEGER NOT NULL,
28     session_id VARCHAR(255) NOT NULL,
29     message_role VARCHAR(50) NOT NULL, -- 'user' 或 'assistant'
30     message_content TEXT NOT NULL,
31     timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
32     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
33     -- 创建复合索引以优化按用户和会话查询历史
34     CREATE INDEX idx_chat_user_session ON ai_chat_history(user_id,
35     session_id)
36 );
37
38 -- 用户收藏表
39 CREATE TABLE IF NOT EXISTS user_favorites (
40     id INTEGER PRIMARY KEY AUTOINCREMENT,
41     user_id INTEGER NOT NULL,
42     course_id VARCHAR(255) NOT NULL,
43     course_name VARCHAR(255) NOT NULL,
44     course_info TEXT, -- JSON格式存储课程详细信息
45     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
45 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
46 UNIQUE(user_id, course_id), -- 确保用户对同一课程只能收藏一次  
47 -- 为用户ID添加索引  
48 CREATE INDEX idx_favorites_user_id ON user_favorites(user_id)  
49 );
```

Listing 1: 完整的数据库初始化脚本

6.2 索引设计策略与优化

索引是提高数据库查询性能的关键。我们的索引设计基于对系统查询模式的分析：

表 1: 数据库索引详细设计

索引位置	索引字段/类型	设计理由与预期效果
users.id	主键索引 (Primary Key)	用于唯一标识用户，是其他表外键引用的目标，自动创建。
users.email	唯一索引 (Unique Index)	用于用户登录时的邮箱快速查找和注册时的重复性校验，由于 UNIQUE 约束自动创建。
timetables.id	主键索引 (Primary Key)	唯一标识课表。
timetables.user_id	普通索引 (Index)	最重要的索引之一 。系统中大量操作需要查询某个用户的所有课表（如加载课表列表）。此索引能显著加速 WHERE user_id = ? 这类查询。
ai_chat_history.id	主键索引 (Primary Key)	唯一标识聊天记录。
ai_chat_history.user_id	普通索引 (Index)	用于查询某个用户的所有聊天记录。
ai_chat_history (user_id, session_id)	复合索引 (Composite Index)	针对核心查询模式优化 。AI 聊天界面通常需要按用户和特定会话来加载历史消息（WHERE user_id = ? AND session_id = ? ORDER BY timestamp）。这个复合索引能完美覆盖此查询条件，效率极高。
user_favorites.id	主键索引 (Primary Key)	唯一标识收藏记录。
user_favorites.user_id	普通索引 (Index)	用于快速查询某个用户的所有收藏课程。
user_favorites (user_id, course_id)	唯一复合索引 (Unique Composite Index)	用于确保收藏唯一性（约束），同时也对按用户和课程查询的语句有加速作用。

索引创建决策：我们选择在创建表的同时显式创建索引（如上 SQL 所示），而不是依赖数据库的自动行为。这样意图更明确，也方便未来进行索引调整。考虑到 SQLite 的特点以及本系统数据量不会爆炸式增长，我们没有过度设计索引，目前的索引方案在查询性能和更新开销之间取得了良好平衡。

6.3 存储参数与引擎选择

数据库引擎：选择 SQLite 3。理由如下：1) **轻量级：**整个数据库是一个单独的文件，无需配置和运行独立的数据库服务器，简化了部署。2) **零配置：**开箱即用，适合课程设计、小型项目或原型开发。3) **事务支持：**支持 ACID 事务，保证在用户保存课表、记录聊天等操作时的数据一致性。4) **足够的性能：**对于本项目预期的用户和数据规模，SQLite 的性能完全足够。5) **兼容性：**几乎所有平台和语言都支持 SQLite。

数据类型映射：

- **INTEGER:** 用于所有 ID 字段和数字字段（如节次）。SQLite 的 INTEGER 支持自增主键。
- **VARCHAR(n):** 用于长度可预测的短文本，如邮箱、姓名、课程 ID 等。我们根据业务逻辑合理估计了长度（如 email 255）。
- **TEXT:** 用于可能较长的文本内容，如课程描述、JSON 数据、聊天内容。SQLite 的 TEXT 类型可以存储任意长度的字符串。
- **DATETIME:** 用于存储时间戳。SQLite 没有原生的 DATETIME 类型，但我们使用 DATETIME 关键字，实际存储为文本（ISO8601 格式），并利用 CURRENT_TIMESTAMP 默认值自动插入当前时间。

字符编码：使用 UTF-8 编码，以完美支持中文课程名称、用户姓名和聊天内容。

6.4 安全性设计深入

数据库安全是多层次的，我们的设计从存储、访问和架构三个方面进行保障：

1. 数据加密存储：

- **密码：**使用 bcrypt 算法对用户密码进行哈希处理并加盐。bcrypt 是专门为密码存储设计的算法，计算缓慢，能有效抵御彩虹表攻击。代码示例：`password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())`。

- **敏感信息**：邮箱、姓名等个人信息以明文存储，因为它们是业务展示所必需的。但系统不存储任何更敏感的信息（如身份证号、手机号）。

2. 访问控制：

- **应用层认证**：所有对用户数据的 API 访问都必须提供有效的 JWT。JWT 中包含了用户 ID，后端在处理每个请求时，都会验证令牌的有效性，并从令牌中提取用户 ID 作为后续数据操作的依据。
- **数据隔离**：所有 SQL 查询都强制包含用户 ID 条件。例如，查询用户课表的语句永远是 `SELECT * FROM timetables WHERE user_id = ?`，这个 `?` 来自 JWT 中的用户 ID，而不是前端传递的参数。这从根本上防止了用户通过修改请求参数来访问他人数据（横向越权）。
- **数据库层约束**：外键约束和级联删除不仅是保证数据完整性的工具，也在一定程度上防止了不一致的、无效的数据关联。

3. 架构安全：

- **SQL 注入防护**：使用参数化查询（Prepared Statements）或 ORM 框架来构建所有 SQL 语句，确保用户输入不会被解释为 SQL 代码。
- **最小权限原则**：连接数据库的后端服务账户，只被授予执行必要 DML 和查询操作的权限，不应具有删除表、修改结构等高风险权限。

7 RAG 与向量检索技术详解

7.1 RAG 技术原理与在本系统的应用

RAG (Retrieval-Augmented Generation) 是一种将信息检索与大语言模型生成相结合的技术范式。其核心思想是：当需要回答一个问题时，不是直接让大语言模型凭空生成，而是先从海量知识库中检索出与问题最相关的文档片段，然后将这些片段作为上下文信息，连同问题一起输入给大语言模型，让它基于这些确切的“证据”来生成答案。

在本课程表管理系统中，RAG 技术的应用场景和流程如下：

1. **触发**：用户在 AI 对话框中输入一个问题，例如：“我周三下午有空，能推荐一些课吗？”
2. **检索**：后端接收到问题后，首先进行“检索”阶段：
 - 从数据库中检索当前用户的课表数据，提取出周三下午已有课程的时间段，从

而计算出“空闲时间”。

- 从内存的课程库中，检索所有在周三下午有课的课程。
- 利用向量检索技术，从课程库中查找与“推荐”、“有意思”、“通识”等语义相关的课程（即使用户没提这些词）。

3. **增强**：将检索到的信息（用户周三下午的空闲时间段、符合时间条件的课程列表、语义相关的课程列表）进行整理，作为“上下文”或“参考文档”。
4. **生成**：精心构造一个 Prompt（提示词），包含以下部分：

系统角色：你是一个专业的大学教务顾问助手。

用户信息：用户当前课表显示，周三下午第5-7节有空闲。

相关课程：检索到以下课程在周三下午开设：1.《电影鉴赏》（5-7节）... 2.《心理学

用户问题：我周三下午有空，能推荐一些课吗？

回答要求：基于提供的课程信息，推荐不时间冲突的课程，并说明理由。你的回答必须是

将这个 Prompt 发送给大语言模型（如 GPT-3.5/4）。

5. **返回**：大模型基于给定的上下文生成一个友好、专业且可操作的答案。例如：“根据您的空闲时间，我为您推荐以下两门课程：...。其中《电影鉴赏》时间完全匹配，可以一键添加。”同时，模型返回的结构化数据会被前端解析，生成“一键添加”按钮。

通过 RAG，AI 的回答不再是泛泛而谈，而是紧密结合了用户的私有数据（课表）和公共数据（课程库），从而实现了真正的个性化推荐，并极大减少了模型“胡编乱造”（幻觉）的可能性。

7.2 向量检索技术的实现细节

向量检索是本系统实现语义搜索（L3）和 RAG 中智能匹配的基础。

7.2.1 向量化模型选择与处理

我们选择 sentence-transformers 库中的预训练模型（如 paraphrase-multilingual-MiniLM-L12-v2）。这个模型能够将中文句子转换为一个 384 维的稠密向量（embedding）。选择它是因为：

- **多语言支持**：完美支持中文课程名称和描述。
- **轻量高效**：模型较小，在 CPU 上也能快速进行推理，适合本系统环境。

- **质量不错**：在语义相似度任务上表现良好，能满足课程检索需求。

处理流程：系统启动时，遍历所有课程，将课程名称和关键信息（如“数据结构与算法”）拼接成一段文本，送入模型，得到该课程的向量 V_{course} ，并存储在内存的一个字典或列表中。

7.2.2 相似度计算与搜索

当用户输入查询 Q （如“编程入门课”）时：

1. 使用同样的模型将 Q 转换为查询向量 V_{query} 。
2. 遍历所有预计算的课程向量 V_{course_i} ，计算 V_{query} 与每个 V_{course_i} 的 **余弦相似度**。

$$\text{similarity} = \frac{V_{query} \cdot V_{course_i}}{\|V_{query}\| \|V_{course_i}\|}$$

余弦相似度的值范围在 $[-1, 1]$ 之间，越接近 1 表示语义越相似。

3. 将所有课程按照相似度得分从高到低排序。
4. 取出 Top K（例如 $K=10$ ）个结果，作为语义搜索的结果返回。

7.2.3 性能优化考虑

为了提高搜索速度，我们采用了以下优化：

- **预计算**：所有课程向量在启动时一次性计算好，避免每次搜索都重复计算。
- **使用 NumPy 进行批量计算**：计算余弦相似度时，利用 NumPy 的广播机制和向量化运算，可以一次性计算查询向量与所有课程向量的相似度，效率远高于循环。
- **阈值过滤**：可以设置一个相似度阈值（如 0.5），只有超过阈值的课程才被认为是相关结果，减少不必要的数据传输。

7.3 三层混合搜索架构的融合策略

如前所述，系统并行执行 L1（关键词）、L2（AI 联想）、L3（向量）三种搜索。如何融合这三种结果至关重要。

7.3.1 并行执行与结果获取

1. **L1 关键词搜索**：在内存中的课程列表里，使用简单的字符串包含检查（如 Python 的 `in` 操作）或正则表达式，找出课程名称或 ID 中包含查询关键词的课程。结果记为 R_1 。
2. **L2 AI 联想搜索**：将用户原始查询发送给一个快速、廉价的 LLM（或大模型的 fast 模式），要求其提取出 3-5 个标准化、相关的关键词。例如，“有什么好玩的课吗？”可能被提取为“趣味”、“通识”、“实践”。然后用这些关键词再次执行 L1 搜索。结果记为 R_2 。
3. **L3 向量语义搜索**：如上节所述，通过向量相似度计算得到结果。结果记为 R_3 。

7.3.2 结果融合算法

我们采用一种加权打分和去重合并的策略：

1. **归一化打分**：为每个结果列表中的课程赋予一个初始分数。L1 结果（精确匹配）分数最高（如 1.0），L2 结果（联想匹配）次之（如 0.7），L3 结果（语义匹配）再次之（如 $0.5 * \text{相似度分数}$ ）。
2. **加权调整**：根据查询意图动态调整权重。例如，如果查询很短，像是关键词（如“CS101”），则增加 L1 的权重；如果查询是长句、口语化，则增加 L3 的权重。
3. **合并与去重**：将三个列表合并，如果同一门课程出现在多个列表中，则取其最高分作为该课程的最终分数。
4. **重排序**：根据最终分数对所有课程进行降序排序。
5. **返回**：取排名靠前的 N 门课程（如 20 门）返回给前端。

这种融合策略兼顾了精确性、召回率和智能化，为用户提供了良好的搜索体验。

7.4 RAG 上下文构建与提示词工程

RAG 的效果很大程度上取决于检索到的上下文质量和提示词的构造。

7.4.1 动态上下文构建

对于不同的用户问题，系统检索不同的上下文：

- **时间相关问题**（如“我周四下午能上什么课？”）：检索用户课表（计算空闲时间）+ 课程库中对应时间段的课程。
- **课程推荐问题**（如“推荐一些数学课”）：检索用户已修/已选课程（避免重复）+ 课程库中所有数学相关课程（通过关键词或向量检索）。
- **冲突检查问题**（如“如果我选这门课，会冲突吗？”）：检索用户当前课表 + 目标课程的详细时间安排。

检索到的原始数据会被整理成一段连贯、清晰的文本描述，作为上下文。

7.4.2 系统提示词设计

系统提示词 (System Prompt) 是指导 AI 行为的核心。我们的提示词包含以下几个部分：

1. 角色设定：你是一位专业、友好、乐于助人的大学教务AI助手。
2. 知识来源：你拥有当前学期的完整课程数据库，并且能够访问当前用户的个人课表信息。
3. 回答原则：
 - 你必须基于我给你提供的课程数据和用户课表信息来回答问题。
 - 如果提供的信息不足以回答问题，请诚实告知用户，不要编造信息。
 - 对于课程推荐、时间冲突等问题，你的回答必须是具体、可操作的。
 - 请用清晰、有条理的方式组织你的回答。
4. 输出格式：你的回答应该包含两个部分：
 - 第一部分：给用户的自然语言回复。
 - 第二部分（可选）：一个JSON格式的结构化数据，用于前端渲染交互式组件。例如，如果是课程推荐，JSON应包含推荐课程的ID、名称和操作指令。
5. 当前上下文：[此处动态插入检索到的上下文信息]
6. 用户当前问题：[此处插入用户的最新问题]

精心设计的提示词将 AI 严格限制在“基于事实的顾问”角色内，确保了回答的准确性和实用性。

8 系统功能实现与数据交互

8.1 用户管理功能的实现

用户管理围绕 `users` 表展开，主要 API 端点包括：

- `POST /api/register`: 接收邮箱、密码、姓名。检查邮箱唯一性，用 `bcrypt` 哈希密码，插入新用户记录，返回成功信息。
- `POST /api/login`: 接收邮箱、密码。根据邮箱查找用户，使用 `bcrypt.checkpw` 验证密码，若成功则使用 `PyJWT` 生成一个包含用户 ID 和邮箱的令牌（设置 30 天有效期），返回给前端。
- `GET /api/profile`: 需要 JWT 认证。从令牌中获取用户 ID，查询并返回用户信息（除密码哈希外）。
- `PUT /api/profile`: 更新用户姓名或头像 URL。

所有返回用户数据的接口都确保只返回当前令牌所属用户的数据，通过 SQL 语句中的 `WHERE id = ?` 实现，问号参数来自 JWT 解码后的用户 ID。

8.2 课程搜索功能的实现

课程搜索是系统的核心功能，其实现结合了内存数据操作和 AI 调用。

- `GET /api/courses`: 返回所有课程列表（或分页返回）。这是基础接口。
- `POST /api/courses/search`: 接收搜索查询字符串。后端启动一个异步任务，并行执行 L1, L2, L3 搜索（L2 可能涉及调用 OpenAI API）。使用 `asyncio.gather` 等待所有结果，然后调用融合算法进行合并排序，最后返回结果列表。

由于课程数据在内存中，搜索速度极快。L2 和 L3 的 AI 相关操作是主要性能瓶颈，需要通过异步、缓存（缓存联想关键词结果）等技术进行优化。

8.3 课表管理功能的实现

课表管理涉及对 `timetables` 表的增删改查，并且需要处理 JSON 格式的课程数据。

- `GET /api/timetables`: 获取当前用户的所有课表摘要（ID、名称、描述等）。
- `POST /api/timetables`: 创建新课表。前端传递课表名称、描述和课程列表（JSON 格式）。后端验证课程 ID 的有效性（是否存在于内存课程库中），然后执行时间冲突检测算法。若无冲突，将课程列表转为 JSON 字符串，连同用户 ID 一起存入数

数据库。

- GET /api/timetables/<id>: 获取指定 ID 课表的详细信息, 包括完整的课程 JSON 数据。
- PUT /api/timetables/<id>: 更新课表。同样需要验证课程和冲突检测。
- DELETE /api/timetables/<id>: 删除课表。由于有 ON DELETE CASCADE 约束, 直接删除即可。

冲突检测算法: 遍历课表中所有课程的所有 Session (时间段), 将每个 Session 转换为一个时间块 (由星期、开始节次、结束节次定义), 然后检查这些时间块之间是否有重叠。算法复杂度为 $O(n^2)$, 但对于单个课表的课程数量 (n 通常小于 10), 效率完全可以接受。

8.4 AI 交互功能的实现

AI 交互是系统智能化的体现, 涉及 ai_chat_history 表和复杂的 RAG 流程。

- POST /api/chat: 核心的聊天接口。接收用户消息和会话 ID (如果开始新会话, 前端生成新的会话 ID)。
- 后端处理流程:
 1. 将用户消息存入 ai_chat_history 表 (role='user')。
 2. 根据会话 ID, 从同一表中取出最近的几条历史消息 (用于维持对话上下文)。
 3. 分析用户消息意图, 触发相应的 RAG 检索流程 (如检索用户课表、相关课程等)。
 4. 构建包含系统角色、检索到的上下文、历史对话和当前问题的完整 Prompt。
 5. 调用 OpenAI ChatCompletion API (或本地部署的模型) 获取 AI 回复。
 6. 解析 AI 回复, 提取自然语言部分和可能的 JSON 结构化数据。
 7. 将 AI 回复存入 ai_chat_history 表 (role='assistant')。
 8. 将自然语言回复和结构化数据一并返回给前端。
- GET /api/chat/sessions: 获取当前用户的所有会话列表。
- GET /api/chat/history?session_id=<id>: 获取指定会话的完整历史消息。

为了提升响应速度, 对用户课表等频繁访问的数据可以进行缓存。同时, 需要设置合理的超时时间和错误处理机制, 确保 AI 服务暂时不可用时, 系统仍能正常运作。

9 数据库优化策略与性能调优

9.1 查询优化实践

针对系统的具体查询模式，我们实施了以下优化：

- **索引覆盖查询**：对于 `SELECT id, name FROM timetables WHERE user_id = ?` 这样的查询，我们在 `user_id` 上建立的索引，使得数据库可以直接从索引中获取所需数据 (`id, user_id, name`)，无需回表查询数据行，速度极快。
- **避免 SELECT ***：在代码中明确指定需要查询的列，减少不必要的数据传输和内存占用，尤其是在 `courses` 字段存储大量 JSON 数据时。
- **合理使用 JOIN**：本系统表关系相对简单，主要查询都是单表查询或通过应用层代码关联（先查用户课表，再根据课表中的课程 ID 去内存课程库查详情），避免了复杂的多表 JOIN，这本身也是一种优化。
- **分页查询**：对于聊天记录、课表列表等可能增长的数据，API 支持分页参数 (`limit, offset`)，避免一次性返回过多数据。

9.2 存储优化策略

- **JSON 字段压缩**：在将课程列表存入 `timetables.courses` 字段前，可以使用 `gzip` 或简单的空格去除进行压缩，减少存储空间。读取时再解压。由于 SQLite 中 TEXT 字段的存储效率已经很高，此优化可根据数据量大小决定是否采用。
- **归档旧数据**：制定数据归档策略。例如，可以定期将超过一年的 AI 聊天历史迁移到单独的归档表中，或者直接导出为文件后从主表删除，保持主表的轻量。
- **定期 VACUUM**：SQLite 在执行大量删除操作后，会产生内部碎片。可以定期（例如每周一次）在系统低峰期执行 `VACUUM` 命令，回收空间并优化数据库文件结构。

9.3 性能监控与调优建议

虽然 SQLite 本身管理简单，但仍需关注性能：

- **启用 WAL 模式**：在并发读写稍多的场景下，可以将 SQLite 的日志模式从默认的“回滚日志”改为“预写式日志” (WAL)。WAL 模式允许多个读操作和一个写操作同时进行，能提升并发性能。命令：`PRAGMA journal_mode=WAL;`。

- **监控慢查询**：可以在 Flask 应用中添加中间件，记录执行时间超过特定阈值（如 200ms）的 SQL 语句及其参数，用于分析优化。
- **使用 EXPLAIN QUERY PLAN**：对于复杂的或性能不佳的查询，使用 EXPLAIN QUERY PLAN 命令查看 SQLite 的执行计划，检查是否用上了预期的索引。
- **连接池**：虽然 SQLite 是文件数据库，但频繁打开关闭连接也有开销。可以使用简单的连接池或确保在一个请求周期内复用数据库连接。

10 数据库维护、备份与安全

10.1 备份策略与实施

数据是系统的核心资产，必须建立可靠的备份机制。

- **全量备份**：每天凌晨自动执行一次全量备份。使用 Python 脚本复制整个 SQLite 数据库文件 (*.db) 到备份目录，文件名加上日期戳。同时备份课程 CSV 文件。
- **备份保留策略**：保留最近 7 天的每日备份，以及最近 4 周的每周一备份。更旧的备份可以删除或转移到成本更低的云存储。
- **备份验证**：定期（例如每周）从备份文件中恢复数据到测试环境，验证备份的完整性和可用性。
- **异地备份**：将备份文件同步到云存储服务（如 AWS S3、阿里云 OSS），实现异地容灾。

10.2 恢复流程与应急预案

制定详细的恢复流程文档，确保在发生数据丢失或损坏时能快速恢复：

1. **确定故障点**：是单用户数据错误、整个表损坏还是数据库文件损坏？
2. **选择备份**：根据故障时间点选择最新的有效备份。
3. **执行恢复**：停止应用服务；用备份文件替换损坏的数据库文件；重启应用服务。
4. **验证恢复**：检查关键功能是否正常，数据是否完整。
5. **数据补录**：如果备份后有新增数据，评估是否需要从日志或其他渠道进行补录（本系统通常可接受少量最新数据的丢失）。

10.3 数据迁移与版本管理

随着系统迭代，数据库结构可能发生变化（增加字段、修改类型等）。需要版本管理。

- **迁移脚本**：每次数据库结构变更，都编写对应的 SQL 迁移脚本（upgrade.sql 和 rollback.sql），并记录版本号。
- **版本控制**：在数据库中创建一个 `schema_versions` 表，记录当前数据库的版本号。应用启动时检查版本号，自动执行需要的迁移脚本。
- **回滚计划**：每次升级前，务必进行全量备份，确保升级失败时可以快速回滚到旧版本。

10.4 安全加固措施

- **文件系统权限**：确保数据库文件（*.db）的读写权限仅限于运行后端服务的账户，防止未授权访问。
- **网络隔离**：确保数据库文件所在的服务器或容器不直接暴露在公网上。所有访问必须通过后端 API。
- **定期审计**：定期审查数据库操作日志（如果开启），检查是否有异常访问模式。

11 系统架构与技术栈总结

11.1 前端技术栈详解

- **Vue 3 (Composition API)**：提供了响应式和组件化的开发体验。使用 Composition API 使得逻辑复用（如课表冲突检测函数）更加清晰。
- **TypeScript**：为所有数据接口（特别是从后端 API 返回的复杂 JSON 结构）定义了严格的类型，如 `Course`, `Timetable`, `AIChatResponse` 等，极大地减少了运行时类型错误，提升了开发效率。
- **Vite**：作为构建工具，提供了极快的热更新速度，优化了开发体验。其按需编译的特性也使得生产环境打包体积更小。
- **CSS Grid / Flexbox**：用于构建课表视图的核心布局技术。CSS Grid 非常适合用来展示以星期为列、节次为行的课程表网格，可以精准控制每个课程块的位置和大

小。

- **Axios**: 用于发起 HTTP 请求到后端 API, 并统一处理 JWT 令牌的携带和过期刷新。

11.2 后端技术栈详解

- **Flask**: 轻量级 Python Web 框架, 用于快速搭建 RESTful API。其扩展生态丰富, 如 Flask-CORS 处理跨域请求。
- **SQLite**: 关系型数据库, 存储用户、课表、聊天记录等结构化数据。
- **PyJWT**: 用于生成和验证 JWT 令牌, 实现无状态认证。
- **LangChain**: 用于编排 AI 工作流。其 RetrievalQA 链等组件可以方便地实现 RAG 模式, 但本项目中我们进行了较多定制化开发。
- **OpenAI API / 本地大模型**: 作为对话生成的核心引擎。通过 API 调用或本地部署的模型提供智能对话能力。
- **Sentence-Transformers**: 用于文本向量化, 是实现语义搜索 (L3) 和 RAG 中智能检索的关键。
- **Pandas / NumPy**: 用于加载和处理 CSV 课程数据, 以及进行高效的向量计算。

11.3 数据流处理全景

整个系统的数据流动形成了一个高效的闭环:

1. **静态数据加载**: 系统启动, CSV 课程文件 → Pandas 解析 → 内存 Course 对象数组 + 预计算向量。
2. **用户请求处理**: 用户在前端操作 → Axios 发送 HTTP 请求 → Flask 路由接收 → JWT 验证和用户身份提取 → 业务逻辑处理 (可能涉及数据库 CRUD 或 AI 调用) → 返回 JSON 响应 → 前端 Vue 组件更新状态和视图。
3. **AI 智能交互**: 用户提问 → 触发 RAG 流程 → 检索用户上下文和课程信息 → 构建 Prompt → 调用大模型 → 解析并存储回复 → 返回结构化和自然语言结果 → 前端渲染对话和可交互卡片。
4. **数据持久化**: 用户关键操作 (保存课表、发送消息) → 通过 SQLAlchemy 或直接 SQL 语句 → 写入 SQLite 数据库文件。

这个数据流设计确保了系统的响应性、智能性和数据持久性。

12 总结与展望

12.1 项目成果总结

本课程表管理系统数据库设计报告，完整呈现了一个融合现代 AI 技术的智能教学平台的后端数据架构。我们成功地将传统的课程管理需求与前沿的 RAG、向量检索技术相结合，设计出了一套既满足基础功能需求，又具备高度智能化和扩展性的数据库方案。

12.1.1 设计亮点回顾

- 清晰的分层设计：**将相对静态的课程数据置于内存实现高速检索，将动态、私有的用户数据置于关系数据库保证持久化和事务安全，这种混合存储策略是性能与功能需求的完美平衡。
- 规范化与实用性的结合：**核心用户相关表严格遵循 3NF，保证了数据一致性。而在课表存储中，出于性能和简化模型的考虑，采用了 JSON 字段，在可控的复杂度内获得了更大的灵活性。
- 深度集成 AI 技术：**数据库设计充分考虑了对 AI 功能的支撑。`ai_chat_history` 表记录了完整的交互上下文，为多轮对话和用户体验分析提供了可能。内存中的课程向量化数据则是实现智能语义搜索的基石。
- 周全的安全考虑：**从密码哈希存储、JWT 认证、到 SQL 查询中的用户 ID 强制过滤，形成了多层次的安全防御体系，有效保护了用户隐私和数据安全。
- 性能导向的优化：**从内存存储课程、精心设计的索引、到向量计算的预加载和批量处理，每一个设计决策都贯穿着对性能的追求，确保了系统在高并发和复杂查询下的流畅体验。

12.1.2 数据库设计价值

本数据库设计不仅支撑了系统当前所有功能（用户管理、智能搜索、课表可视化、AI 对话），其良好的结构也为未来可能的扩展预留了空间。例如，可以轻松增加课程评分表、教师信息表、选课记录表等，而无需对现有核心结构进行伤筋动骨的修改。

12.2 挑战与解决方案

在设计和实现过程中，我们也遇到并克服了一些挑战：

- **挑战一：复杂排课信息的解析。**CSV 中的排课字符串格式不规则。**解决方案：**开发了基于正则表达式的健壮解析函数 `parse_schedule_string`，能够处理多教师、多时间段等复杂情况。
- **挑战二：混合搜索的融合策略。**如何公平、有效地合并三种不同搜索技术的结果？**解决方案：**设计了基于加权打分和动态权重调整的融合算法，在实践中不断调整参数以达到最佳效果。
- **挑战三：RAG 上下文的实时性与准确性。**如何快速检索出最相关的用户上下文？**解决方案：**根据问题类型（时间、推荐、冲突）预定义检索模板，并利用向量检索辅助查找语义相关的课程，确保了上下文的针对性和完整性。

12.3 未来展望与扩展方向

尽管当前设计已经相当完善，但仍有可以探索和优化的方向：

- **向量数据库集成：**当课程数量极大（如超过 10 万门）时，内存向量检索可能遇到瓶颈。未来可以考虑集成专业的向量数据库（如 Milvus, Pinecone 或 PostgreSQL 的 pgvector 扩展），实现亿级向量的高效近似最近邻搜索。
- **缓存策略优化：**可以引入 Redis 等内存数据库，缓存热门课程、用户课表、AI 问答对等，进一步减轻数据库压力和提升响应速度。
- **数据分析和推荐系统：**基于积累的用户收藏、课表历史和 AI 交互数据，可以构建更精准的协同过滤或知识图谱推荐系统，实现“猜你喜欢”功能。
- **微服务化架构：**随着功能复杂化，可以将用户服务、课程搜索服务、AI 服务等拆分为独立的微服务，每个服务拥有自己的数据库，通过 API 网关进行通信。数据库设计也需要相应地进行垂直拆分。
- **多数据库支持：**通过使用 SQLAlchemy 等 ORM 工具，可以相对容易地将底层数据库从 SQLite 迁移到更强大的 PostgreSQL 或 MySQL，以支持更高的并发和更复杂的查询。

12.4 结语

综上所述，本课程表管理系统的数据库设计是一个紧密结合业务需求、充分利用现代技术、并充分考虑性能、安全与扩展性的优秀案例。它不仅仅是一个存储数据的容器，更是整个系统智能化和高效运行的基石。通过本次设计实践，我们深刻体会到，一个好的数据库设计需要在规范化与反规范化、通用性与特异性、性能与功能之间做出精妙的权衡。本报告所呈现的设计方案和思考过程，希望能为类似的教育科技或智能化管理系统的数据库设计提供有益的参考和借鉴。